Mansoura University
Faculty of Computers and Information Sciences
Department of Computer Science
First Semester- 2023-2024

[CS212P/IT212] Computer Organization and Architecture

Grade : 2nd General / 3rd Programs

Eng. Esraa Salah

ARITHMETIC OPERATION OF BINARY NUMBER SYSTEM

- Binary Numbers have base of 2 digits 0 and1.
- Arithmetic operation of binary number:
  - Addition.
  - Subtraction.
  - Multiplication.
  - Division.

BINARY ADDITION EX.

- **Rule :**

0	0	1	1
+0	+1	+0	+1
0	1	1	10

1-5. Add the following 8-bit binary numbers:

(a) 00101101 + 00000101

(b) 00110110 + 00111001

(c) 00101111 + 00000001

(d) 01111111 + 01010101

BINARY SUBTRACTION

- **Rule:**

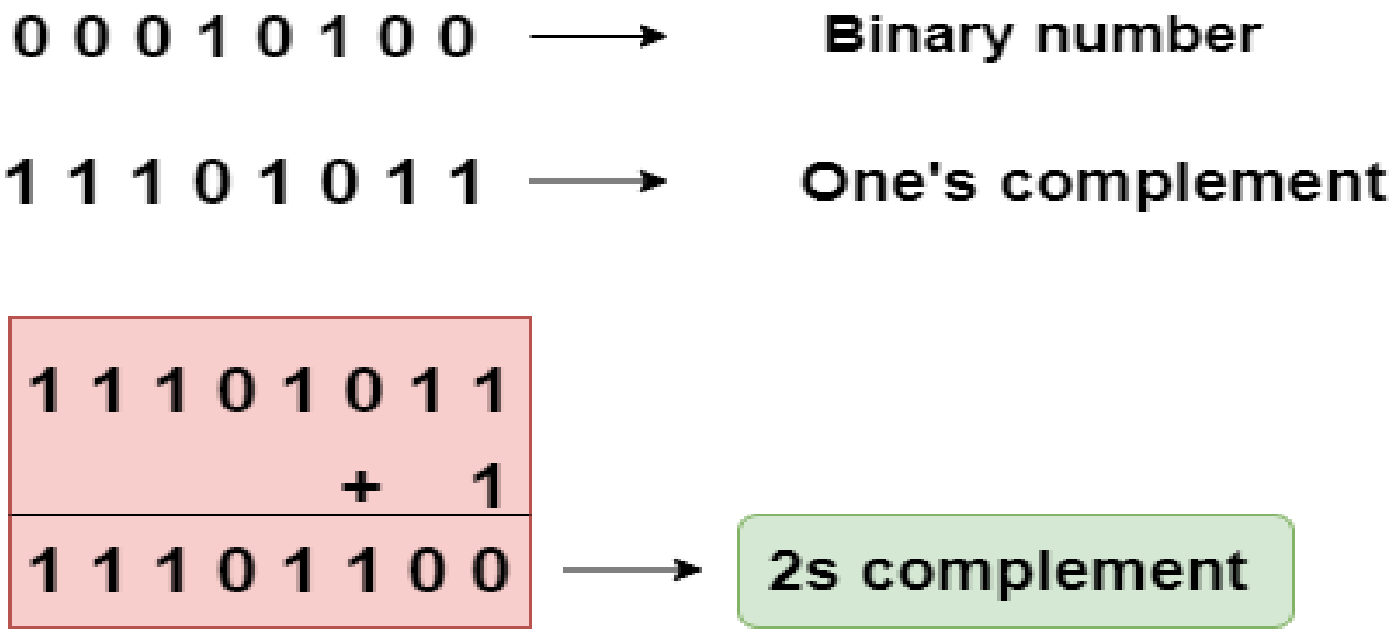
0	0	1	1
- 0	- 1	- 0	- 1
0	1	1	0

(Borrow 1 from the next high order digit)

TWO'S COMPLEMENT

1-7. Provide the two's complement of the following binary numbers: (a) 00110110, (b) 00111101, (c) 01111100, (d) 00000000.

TWO'S COMPLEMENT



CONT.

1-8. Convert the following negative binary numbers into positive binary values: (a) 11001100. (b) 10110111, (c) 10101010, (d) 11111111.

BINARY SUBTRACTION AND TWO'S COMPLEMENT

15		00001111
	-	-
8		00001000
		00000111
7		

BINARY SUBTRACTION AND TWO'S COMPLEMENT

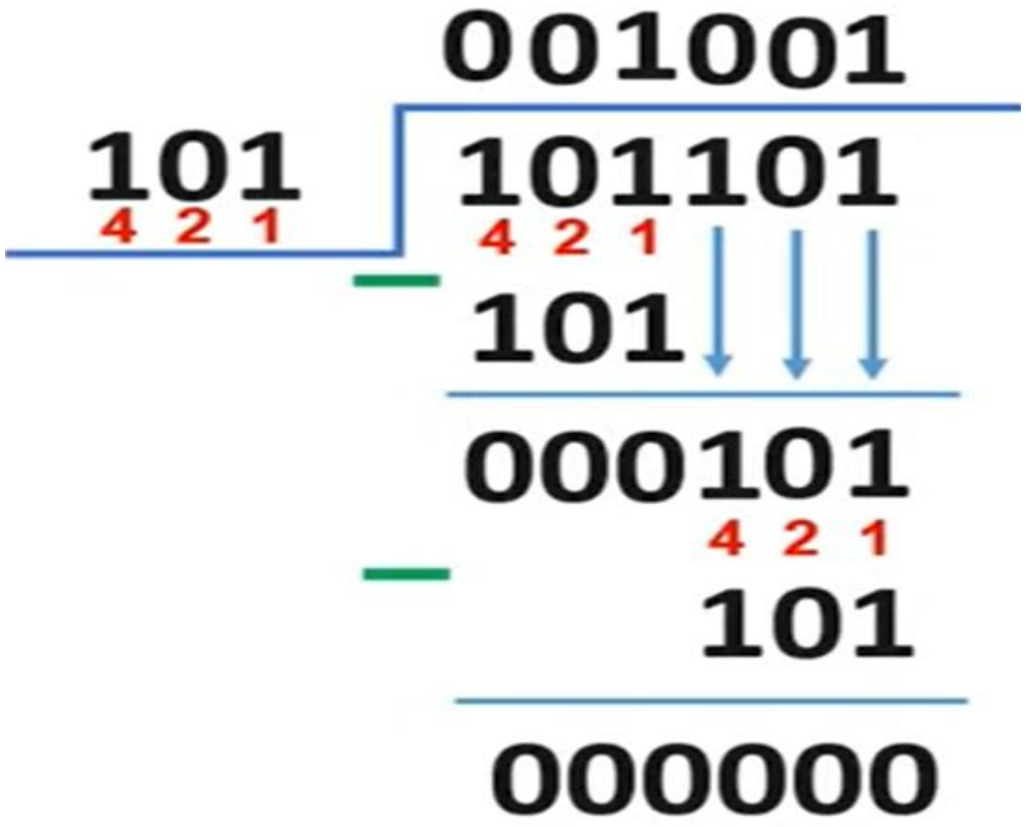
15		00001111
-		-
8		00001000
7		00000111

00001111	
-	
00001000	
11111000	
+	
00001111	
1 00000111	

BINARY MULTIPLICATION

$$
\begin{array}{cccccccc}
 & & & & 1 & 0 & 1 & 1 \\
 & & & \times & 1 & 1 & 0 & 0 \\
\hline
 & & & & 0 & 0 & 0 & 0 \\
 & & & 0 & 0 & 0 & 0 & \\
 & & 1 & 0 & 1 & 1 & & \\
 & 1 & 0 & 1 & 1 & & & \\
\hline
1 & 0 & 0 & 0 & 0 & 1 & 0 & 0
\end{array}
$$

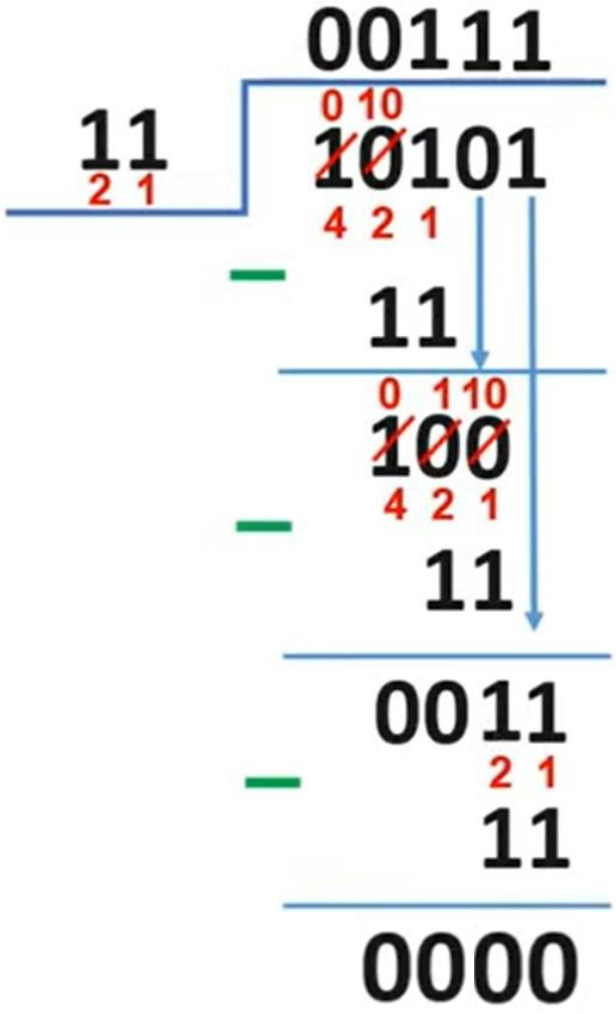
BINARY DIVISION



الخطوات

÷ → * → - → ↓

BINARY DIVISION



الخطوات

÷ → * → - → ↓

UTF (UNICODE TRANSFORMATION FORMAT)

- UTF stands for **Unicode Transformation Format**. It refers to a family of encoding standards that can represent every character in the Unicode character set, which includes all the characters from various writing systems, symbols, and emojis across the world.
- Each UTF encoding defines a different way to convert a Unicode code point (the numerical value representing each character) into binary data (bytes) for use in computers and communication systems.
- The most common UTF encodings are UTF-8 ,UTF-16,UTF-32.

UTF (UNICODE TRANSFORMATION FORMAT)

- The most common UTF encodings are:

1. **UTF-8**: Uses 1 to 4 bytes to encode each character. It is the most widely used encoding, especially on the web, because it's efficient for texts primarily composed of ASCII characters while supporting the entire range of Unicode.

   ASCII characters like "Hello, world!" are encoded as-is.

1. **UTF-16**: Uses 2 or 4 bytes to encode each character. It is often used in environments where many non-ASCII characters (e.g., Asian scripts) are common. UTF-16 represents most common characters with 2 bytes, while some characters require 4 bytes.
2. **UTF-32**: Uses a fixed 4 bytes to encode every character. It's the simplest to decode, as every character takes the same amount of space, but it is the most space-inefficient because even simple characters (like those in ASCII) use 4 bytes.

UTF (UNICODE TRANSFORMATION FORMAT)

Encoding Structure:

Unicode Range	Number of Bytes in UTF-8	Structure	Byte Range
0x0000 - 0x007F	1 byte	0xxxxxxx	0 - 127
0x0080 - 0x07FF	2 bytes	110xxxxx 10xxxxxx	128 - 2047
0x0800 - 0xFFFF	3 bytes	1110xxxx 10xxxxxx 10xxxxxx	2048 - 65535
0x10000 - 0x10FFFF	4 bytes	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx	65536 - 1114111

UTF (UNICODE TRANSFORMATION FORMAT) EXAMPLE

- **Character "A" (U+0041):**
- In **Unicode**: U+0041 → $41_{16} = 65_{10}$
- In **UTF-8**:
  - Since the value is less than **0x007F**, this character will be represented by **1 byte**.
  - Result: **01000001** (or **0x41** in hexadecimal, same as ASCII).

UTF (UNICODE TRANSFORMATION FORMAT) EXAMPLE

- **Character "é" (U+00E9):**
- In **Unicode**: U+00E9 → $E9_{16} = 223_{10}$
- In **UTF-8**:
  - This value falls in the range **0x0080 - 0x07FF**, so it requires **2 bytes**.
  - The breakdown is as follows:
    - First byte: **11000011** (or **0xC3**)
    - Second byte: **10101001** (or **0xA9**)
  - Result: **11000011 10101001**

    0xC3 0xA9

UTF (UNICODE TRANSFORMATION FORMAT) EXAMPLE

- **3. Character "𐍈" (U+10348):**
- In **Unicode**: U+10348 → **$10348_{16} = 66376_{10}$**
- In **UTF-8**:
  - This value falls in the range **0x10000 - 0x10FFFF**, so it requires **4 bytes**.
  - The breakdown is as follows:
    - First byte: **11110000** (or **0xF0**)
    - Second byte: **10010000** (or **0x90**)
    - Third byte: **10001101** (or **0x8D**)
    - Fourth byte: **10001000** (or **0x88**)
  - Result: **11110000 10010000 10001101 10001000**

    0xF0 0x90 0x8D 0x88

UTF (UNICODE TRANSFORMATION FORMAT)

- If the first byte starts with **0**, it means the character is **1 byte** long (ASCII).
- If the first byte starts with **110**, it means the character is **2 bytes** long.
- If the first byte starts with **1110**, it means the character is **3 bytes** long.
- If the first byte starts with **11110**, it means the character is **4 bytes** long.

THANKS ♥